

Salesforce in Teams

Authentication Reference

Two login mechanisms, how they fit together, every setting they depend on, and every real issue hit (and fixed) building them.

Part A - OAuth 2.0 Authorization Code + PKCE (interactive popup)

Part B - Microsoft Teams SSO + OAuth 2.0 Token Exchange (silent)

2026-09-27

0. Overview

The "Salesforce in Teams" personal tab (teamsapp-salesforce/, app id 3aed85f6-5208-454a-8c48-9ad5fbd14057) authenticates each person with their own real Salesforce access - never a shared integration user, never a relay holding anyone's token. Two mechanisms exist, tried in this order every time the tab needs a session:

Order	Mechanism	What it feels like
1st	Teams SSO + Token Exchange (Part B)	Completely silent - no popup, no click, no visible sign-in step at all.
2nd (fallback)	OAuth 2.0 Authorization Code + PKCE (Part A)	A "Sign in" button, then a real popup showing Salesforce's login page.

The fallback is automatic and silent to the code path, not just a manual option: `ensureSfSession()` in `salesforceTabHome.js` tries `loginWithTeamsSso()` first; on ANY failure there (missing setup, a revoked consent, an Azure AD outage, anything) it surfaces a real "Sign in" button instead, which runs the Part A flow. This means Part B can be adopted or rolled back at any time by changing only Salesforce/Azure AD configuration - no code redeploy, no Teams app reinstall - the same "no redeploy needed to revert" pattern already used elsewhere in this project (e.g. the Case Swarm chat bridge's three delivery modes).

Why not just build Part B and skip Part A entirely?

Part A was built first and reached working end-to-end before Part B existed - it remains the safety net. Part B needs meaningfully more setup (a second Azure AD app, an Apex class doing real JWT signature validation, a Salesforce Permission Set assignment for every user) and depends on infrastructure a given org may not have finished configuring. Keeping Part A as the automatic fallback means the tab never gets stuck if any piece of Part B's setup is missing, misconfigured, or not yet done for a new person.

A. Part A - OAuth 2.0 Authorization Code + PKCE (interactive popup)

A.1 Why this design (the constraint that forced it)

A Microsoft Teams personal tab is rendered as an `<iframe>`. Salesforce's own login page refuses to render inside any iframe, from any origin - proven live against this exact org, not assumed:

```
$ curl -sD - -o /dev/null https://mylightningapp-dev-dev-ed.my.salesforce.com/  
X-FRAME-OPTIONS: DENY  
Content-Security-Policy: frame-ancestors 'none'
```

Checked the "maybe it's fine once already logged in" loophole too, not just the login page - an authenticated Lightning page was tested the same way (via `/secur/frontdoor.jsp`):

```
X-FRAME-OPTIONS: SAMEORIGIN  
Content-Security-Policy: frame-ancestors 'self'
```

'self' still excludes Teams (a different origin) - there is no sequencing trick that lets Salesforce's UI or login screen render directly inside the Teams tab's iframe, logged in or not. This is why the login step has to happen in a real, separate, top-level browser window (a popup), not a redirect of the tab itself.

Not a workaround - a documented Teams pattern

`microsoftTeams.authentication.authenticate({url, width, height})` is the Teams SDK's own generic OAuth popup helper, built for exactly this situation (any third-party identity provider). It is a completely different API from Teams SSO's `getAuthToken()` (Part B).

The popup also deliberately shows full browser chrome, including the address bar with Salesforce's/Microsoft's raw URL visible - Microsoft's intentional anti-phishing design for any third-party OAuth popup in Teams, so a malicious Teams app can never show a chrome-less, spoofed "Microsoft login" page. Every third-party OAuth Teams integration (Dropbox, Zoom, Asana, etc.) looks the same way. Not a defect.

A.2 Architecture at a glance

The relay's only job is serving two static pages (`salesforceTabHome.js`, `salesforceAuthCallback.js`). No auth or data logic runs in Azure at all - every OAuth step and every Salesforce data call happens in the browser, using that person's own token ("Option B" in the project plan: the fastest path to real per-user data, at the cost of the access token living in the browser rather than behind a server).

A.3 Sequence - first-time login

1. Tab loads

`salesforceTabHome.js` checks `localStorage` (key `sfUserSession`) for a saved session. Nothing is there yet.

2. Teams SSO is tried first and fails (or isn't set up)

`ensureSfSession()` always tries `loginWithTeamsSso()` before ever considering the popup - see Part B. This step assumes that attempt failed or Part B isn't configured in this environment.

3. A real "Sign in" button appears

`promptInteractiveSignIn()` replaces the loading status with a button. The popup is deliberately NOT opened automatically here - see the `FailedToOpenWindow` issue in the troubleshooting log below for why.

4. PKCE pair generated (on click)

randomPkceVerifier() makes a random 64-byte value (code_verifier); pkceChallengeFor() SHA-256-hashes it and base64url-encodes the result (code_challenge). The verifier is stashed in localStorage under sfPkceVerifier so the popup (a different page load) can read it back in step 7.

5. Popup opens

microsoftTeams.authentication.authenticate() opens Salesforce's real /services/oauth2/authorize URL (client_id, redirect_uri, code_challenge, scope) in a Teams-managed popup - a real top-level browser context, not the tab's iframe. Called synchronously from the button's click handler, a genuine user gesture, so browsers don't block it.

6. Salesforce's login page loads in the popup

The org's login page has the Azure_AD_SSO Auth Provider enabled, so it shows "Login with Microsoft" - the person is already signed into Microsoft (via Teams), so this resolves in at most one "Continue as ____" click, never a typed Salesforce username/password.

7. Salesforce resolves the identity, popup exchanges the code itself

AzureAdSsoRegistrationHandler.cls matches the Microsoft identity's email/UPN against a Salesforce User's Azure_AD_Email_Id__c field. Salesforce redirects the popup to salesforceAuthCallback.js with an authorization code. That page - not the tab - reads the verifier from localStorage and does the full code-for-token exchange itself (POST /services/oauth2/token, grant_type=authorization_code, no client secret - PKCE is the proof of possession instead), then calls microsoftTeams.authentication.notifySuccess(JSON.stringify(session)) with the complete session.

8. Session handed back and saved

The tab's authenticate() promise resolves with that exact JSON string. loginWithPopup() parses it directly and calls saveSfSession() - localStorage's own sfUserSession key. See the storage-partitioning issue below for why this handoff is NOT a shared-storage read.

9. Data loads

loadListView() calls Salesforce's UI API directly - fetch(session.instanceUrl + path, {headers: {Authorization: 'Bearer ' + accessToken}}) - and renders the person's own records.

A.4 Sequence - every visit after that (no popup)

ensureSfSession() (called by sfGetWithRetry() before every UI API request) reads the stored session and uses it optimistically - it does not pre-check expiry, since Salesforce doesn't return a usable expires_in for this flow. Whether the token is still fresh only matters once a real request is made:

1. Session found

loadSfSession() returns the saved session - used immediately, no popup shown.

2. Access token still valid

sfGetOnce() succeeds normally - the common case for anyone who used the app recently.

3. Access token expired

Salesforce returns 401. sfGetWithRetry()'s catch handler calls refreshAccessToken(refreshToken) - a plain POST with grant_type=refresh_token, no popup at all.

4. Refreshed session saved, request retried

The new access_token is saved (refresh_token isn't rotated by default) and the original call retried once, transparently.

5. Only if the refresh itself fails

(refresh token expired/revoked) the session is cleared and the interactive "Sign in" prompt (step 3 of A.3) reappears - the only case after first login where it comes back.

Token lifetimes

Access token: short-lived per the org's Session Settings - irrelevant to the person, refreshed silently. Refresh token: refreshTokenPolicyType=SpecificLifetime, 365 Days on the Salesforce_In_Teams app - comfortably covers normal gaps in usage. Also invalidated early by an admin revoking access, a password/MFA change, or the browser clearing storage (private/incognito windows wipe it on close).

A.5 De-duping concurrent logins

loadListView() and loadAppNavObjectTabs() fire several Salesforce calls in parallel on first load. With no stored session yet, each one independently seeing "no session" would otherwise open its OWN popup/SSO attempt at the same moment - real, reproduced bugs (see the troubleshooting log). Fixed with a single shared in-flight promise:

```
var pendingLogin = null;
function ensureSfSession() {
  var session = loadSfSession();
  if (session && session.accessToken) return Promise.resolve(session);
  if (!pendingLogin) {
    pendingLogin = loginWithTeamsSso().then(onSuccess, onSsoFailure);
  }
  return pendingLogin; // every concurrent caller awaits the SAME promise
}
```

A.6 Where each piece lives in the code

Function (salesforceTabHome.js)	Role
saveSfSession / loadSfSession / clearSfSession	Read/write the session object to/from localStorage under sfUserSession.
randomPkceVerifier / pkceChallengeFor / base64UrlEncode	PKCE pair generation via the Web Crypto API (crypto.getRandomValues, crypto.subtle.digest).
loginWithPopup()	Builds the /services/oauth2/authorize URL, stashes the verifier, opens the popup, and receives the finished session from notifySuccess().
refreshAccessToken(refreshToken)	POSTs grant_type=refresh_token - the silent-refresh path.
ensureSfSession()	Tries loginWithTeamsSso() (Part B) first; on failure, marks needsInteractiveSignIn instead of auto-opening a popup.
signInInteractively() / promptInteractiveSignIn()	The real "Sign in" button and its click handler - the only place loginWithPopup() is ever actually called.
sfGetOnce(session, path) / sfGetWithRetry(path)	The per-user Salesforce call, plus refresh-then-retry and re-prompt-as-last-resort logic. Every UI API call goes through sfGetWithRetry.
loadListView(objectApiName, listViewApiName)	Calls ui-api/list-info, ui-api/list-records, and a ListView SOQL query, and renders the table for any object.

relay/src/functions/salesforceAuthCallback.js - the popup's registered redirect target. Does the FULL code-for-token exchange itself (moved here specifically to fix the browser-throttling issue below), then calls notifySuccess()/notifyFailure(). Still deliberately minimal otherwise, matching this project's convention that the relay carries no business logic beyond this.

A.7 Salesforce-side configuration reference

Auth Provider: Azure_AD_SSO

Field	Value
-------	-------

Field	Value
Provider Type	Open ID Connect (not the built-in "Microsoft" type, not the legacy "Microsoft Access Control Service" type)
Authorize Endpoint URL	https://login.microsoftonline.com/80040ab0-138a-4eaa-8a1c-68e4f0465151/oauth2/v2.0/authorize
Token Endpoint URL	https://login.microsoftonline.com/80040ab0-138a-4eaa-8a1c-68e4f0465151/oauth2/v2.0/token
User Info Endpoint URL	https://graph.microsoft.com/oidc/userinfo (required - a real login failed with No_Endpoint_URL until set; pulled from this tenant's own /v2.0/.well-known/openid-configuration, not guessed)
Token Issuer	https://login.microsoftonline.com/80040ab0-138a-4eaa-8a1c-68e4f0465151/v2.0
Default Scopes	openid profile email
Execution User	Required once a Registration Handler is set - runs the handler's SOQL query
Registration Handler	AzureAdSsoRegistrationHandler
Callback URL (auto-generated)	https://mylightningapp-dev-dev-ed.my.salesforce.com/services/authcallback/Azure_AD_SSO - must exactly match the Redirect URI registered in Azure AD

Registration Handler: AzureAdSsoRegistrationHandler.cls

Runs on first login via the Auth Provider (later logins call updateUser() instead, once Salesforce auto-creates a ThirdPartyAccountLink). Matches the incoming identity's email/username against User.Azure_AD_Email_Id__c - the same field this org uses everywhere else to bridge a Microsoft/Teams identity to a Salesforce User. Throws a clear NoLinkedUserException if no match is found - never silently fails or auto-provisions a new user.

External Client App: Salesforce_In_Teams

Field	Value
Client type	Public (isConsumerSecretOptional = true) - no secret can be safely kept in browser-delivered code
PKCE	Required (isPkceRequired = true)
OAuth Scopes	Api, RefreshToken, OpenID
Callback URL	https://case-swarm-relay-sfchatsync-h9hbhzf9eagdbkd6.canadacentral-01.azurewebsites.net/api/salesforceAuthCallback
Consumer Key	3MVG9G9pzCUSkzZsud2BdW9TWBEopIncNXHW02MLEybqDY0coXFBnmBujduJ2Du59lbZXigmCVI4V91wkBg0q (not a secret - sent openly by design)
Permitted Users	AdminApprovedPreAuthorized - "Admin approved users are pre-authorized" (see Part B section B.7 - changed from the original AllSelfAuthorized once Token Exchange required it; a Permission Set granting access must be assigned to every real user)
Refresh token lifetime	SpecificLifetime, 365 Days

A CorsWhitelistOrigin entry for the relay's own origin, plus the separate org-wide "Enable CORS for OAuth endpoints" checkbox (Setup -> CORS), were both required - the whitelist alone covers regular UI API/REST calls but not /services/oauth2/token specifically.

A.8 Troubleshooting log (real issues hit while building this)

Symptom	Cause	Fix
Generic "Problem Logging In" page, no detail; ErrorCode=No_Endpoint_URL	Registration Handler / Auth Provider failure with no LoginHistory row created	Set a debug-log TraceFlag on both the Auth Provider's Execution User and the Automated Process user, then retried, to surface the real error

Symptom	Cause	Fix
"Select an execution user to run the registration handler"	OpenID Connect Auth Providers require a User Info Endpoint URL, which some field references omit	Set it to this tenant's real userinfo_endpoint from its own /v2.0/.well-known/openid-configuration
Login popup never opens at all (Teams Desktop only)	A Registration Handler requires an Execution User to be set on the Auth Provider	Set executionUser to the admin account
Teams rejects a re-uploaded app package: "needs a new app version number"	Teams requires manifestVersion's version field to strictly increase on every re-upload	Bump the version field before re-zipping and re-uploading
Update appears to persist, then reverts (Desktop only)	Desktop Teams' embedded browser engine caches tab content aggressively	Fully quit Teams Desktop (system tray -> Quit) and reopen
THREE popups open at once on a first-ever load (private windows especially)	loadListView() fires 3 parallel calls; each independently saw "no session" and called loginWithPopup(), racing on the shared sfPkceVerifier localStorage key	De-dupe: only the first caller starts the login; every concurrent caller awaits that same in-flight promise (see A.5)
Popup completes cleanly, but the tab still says "Could not load cases right now" - private/incognito windows specifically	Chrome's third-party storage partitioning (enabled first in Incognito, ahead of general rollout) puts the popup's own storage and the iframe's storage in different, mutually invisible buckets	Stop relying on a shared-storage read entirely - pass the finished session as the string argument to notifySuccess(), which Teams delivers back as authenticate()'s resolved value over its own postMessage bridge
"FailedToOpenWindow" when falling back to the popup after a failed silent SSO attempt	Browsers only reliably allow window.open()-based popups triggered synchronously by a real user click; an async SSO attempt in front of it pushes the popup call too far from any real gesture	Never auto-open the popup after an async failure - show a real "Sign in" button and only call the popup from that button's own click handler (A.3, step 3)

B. Part B - Microsoft Teams SSO + OAuth 2.0 Token Exchange (silent)

B.1 What this adds over Part A

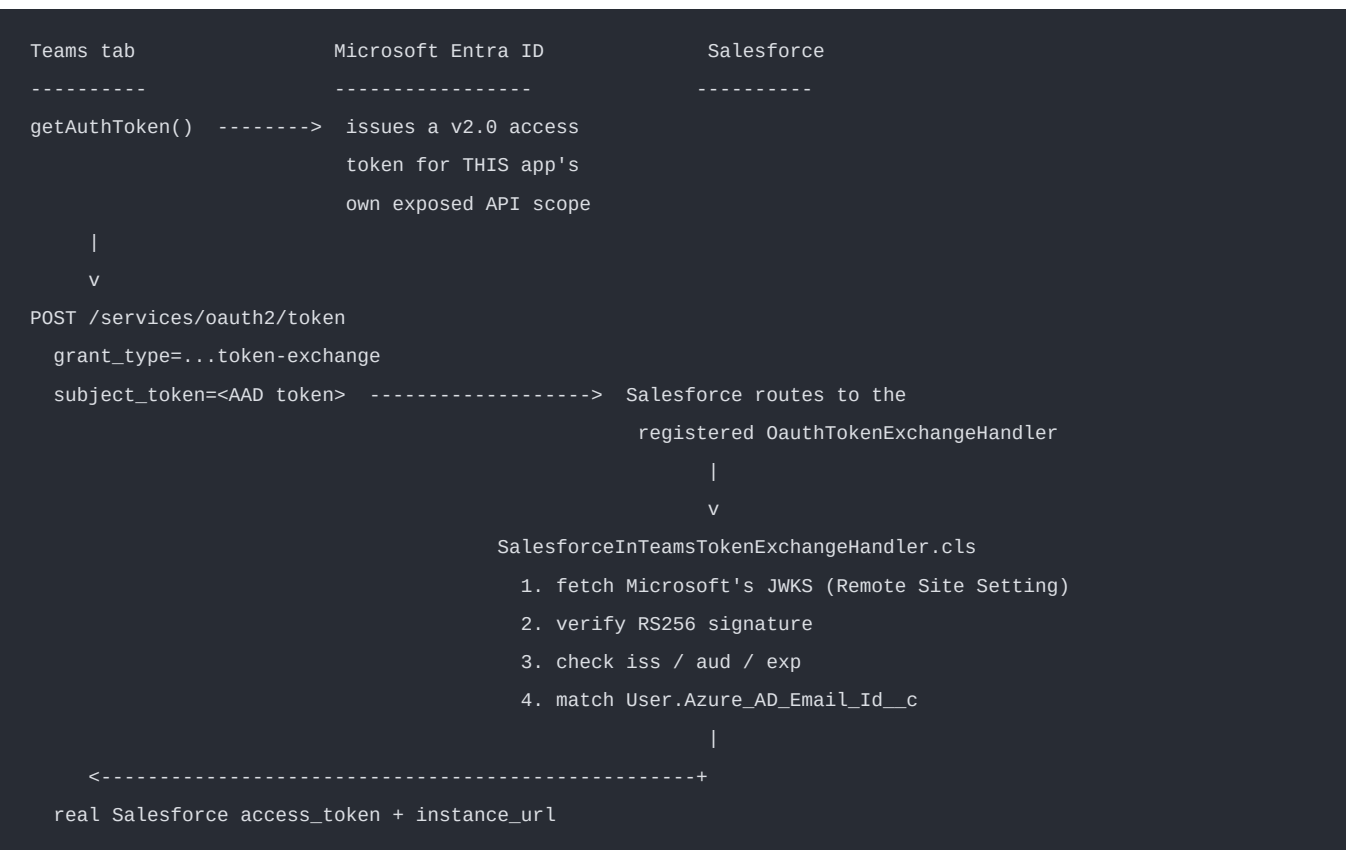
Part A works, but every login (the first one, and any time a refresh token expires or is revoked) shows a popup with Salesforce's own login page in it - expected Teams behavior, but still a visible step. Part B removes that entirely when it succeeds: `microsoftTeams.authentication.getAuthToken()` hands back a signed Azure AD token for whoever is already signed into Teams, with zero user interaction, and Salesforce's OAuth 2.0 Token Exchange flow trades that token directly for a real per-user Salesforce session - no popup, no click, nothing visible at all.

The verification happens INSIDE Salesforce, not in the browser

The Teams tab never validates the Azure AD token itself - it just relays it to Salesforce's token endpoint. A new Apex class, `SalesforceInTeamsTokenExchangeHandler`, does full RS256/JWKS signature verification plus issuer/audience/expiry checks server-side, the same discipline `TeamsBotJwtValidator` already uses for a different issuer (Bot Framework) in the Case Swarm feature. Deliberately not reused/shared code - this project's convention is to copy a proven pattern across features rather than couple them.

B.2 Architecture at a glance

Still no relay in the auth path - the tab calls Teams SSO and Salesforce's token endpoint directly, exactly like Part A calls Salesforce's `authorize/token` endpoints directly. The only new server-side component is the Apex handler, which runs inside Salesforce itself as part of its own OAuth machinery.



B.3 Sequence - silent sign-in (success path)

1. Tab loads, no stored session

`ensureSfSession()` calls `loginWithTeamsSso()` before considering the popup at all.

2. Teams issues an Azure AD token silently

`microsoftTeams.authentication.getAuthToken()` resolves with a signed v2.0 access token for the "Salesforce In Teams SSO" Azure AD app's own exposed scope - no popup, no consent screen, since the two well-known Teams client ids are pre-authorized on that scope (B.4).

3. Token exchanged with Salesforce

A POST to `/services/oauth2/token` with `grant_type=urn:ietf:params:oauth:grant-type:token-exchange`, `subject_token=<the AAD token>`, `subject_token_type=...access_token`, and the same `client_id` as Part A.

4. Salesforce validates and resolves the identity

`SalesforceInTeamsTokenExchangeHandler.validateIncomingToken()` fetches Microsoft's JWKS, verifies the RS256 signature, checks `iss/aud/exp`, then `getUserForTokenSubject()` matches the token's email/UPN against `User.Azure_AD_Email_Id__c` - the exact same field and convention as Part A's Registration Handler.

5. Session saved, data loads

`loginWithTeamsSso()` saves `{accessToken, refreshToken, instanceUrl}` exactly like Part A's session shape - everything downstream (`sfGetWithRetry`, `loadListView`, etc.) is identical either way.

B.4 Azure AD-side configuration reference

Setting	Value / requirement
App registration	A second, separate Azure AD app ("Salesforce In Teams SSO") - not the <code>Azure_AD_SSO</code> Auth Provider's app, and independent of the top-level Teams manifest id (confirmed: they don't need to match)
Expose an API - Application ID URI	<code>api://case-swarm-relay-sfchatsync-h9hbhzf9eagdbkd6.canadacentral-01.azurewebsites.net/b186fa34-7fdf-48be-80c3-7e736eb501b3</code> - MUST include the tab's actual hostname, not just a bare client id (see B.7)
Scope	<code>access_as_user</code> , <code>admins</code> and <code>users</code> can consent
Authorized client applications on that scope	<code>1fec8e78-bce4-4aaf-ab1b-5451cc387264</code> (Teams desktop/mobile) and <code>5e3ce6c0-2b1f-4285-8d4b-75ee78787346</code> (Teams web) - both required for silent consent
Manifest	"api": { "requestedAccessTokenVersion": 2 } - the modern field name/location; the Azure Portal UI still labels it <code>accessTokenAcceptedVersion</code> , but the underlying manifest field moved under <code>api</code>
<code>teamsapp-salesforce/manifest.json</code>	<code>webApplicationInfo</code> : { id: "<client id>", resource: "<the Application ID URI above>" }

B.5 Salesforce-side configuration reference

Setting	Value / requirement
<code>ExtlCIntAppGlobalOAuthSettings.isTokenExchangeEnabled</code>	true (on <code>Salesforce_In_Teams</code>)
<code>ExtlCIntAppOAuthConfigurablePolicies.isTokenExchangeFlowEnabled</code>	true
<code>ExtlCIntAppOAuthConfigurablePolicies.permittedUsersPolicyType</code>	<code>AdminApprovedPreAuthorized</code> (Setup label: "Admin approved users are pre-authorized") - required for a non-interactive flow; see B.7
<code>OAuthTokenExchangeHandler</code> metadata	<code>Salesforce_In_Teams_TokenExchange</code> - points at <code>SalesforceInTeamsTokenExchangeHandler</code> , <code>isAccessTokenSupported=true</code> , everything else (id token, JWT, refresh token, SAML2, user/contact creation) false
"Enable New App" association	Setup -> Token Exchange Handlers -> the handler -> Enable New App -> select <code>Salesforce_In_Teams</code> , set a Run As execution user - a Setup-UI-only step with no equivalent deployable metadata field

Setting	Value / requirement
Permission Set + assignment	An empty Permission Set (Salesforce_In_Teams) listed on the app's policy (commaSeparatedPermissionSet); must be assigned to every real user of this tab once Permitted Users is admin-approved
Remote Site Setting	Azure_AD_JWKS -> https://login.microsoftonline.com (the JWKS fetch in fetchPublicKey() is a real outbound Apex callout and is blocked without this, same as Bot_Framework_Keys for a different domain)

B.6 SalesforceInTeamsTokenExchangeHandler.cls

Extends Auth.OAuth2TokenExchangeHandler. Self-contained rather than reusing TeamsBotJwtValidator.cls (a different issuer, a different feature) - the JWKS/RSA-from-JWK verification logic is the same proven pattern, copied rather than shared.

```
global override Auth.TokenValidationResult validateIncomingToken(
    String appDeveloperName, Auth.IntegratingAppType appType,
    String incomingToken, Auth.OAuth2TokenExchangeType tokenType) {
    try {
        Map<String, Object> claims = validateAndDecode(incomingToken); // RS256/JWKS + iss/aud/exp
        ... build Auth.UserData from claims.preferred_username/upn/email/name/oid ...
        return new Auth.TokenValidationResult(true, null, userData, incomingToken, tokenType, null);
    } catch (Exception e) {
        return new Auth.TokenValidationResult(false); // Salesforce never surfaces WHY to the caller
    }
}

global override User getUserForTokenSubject(Id networkId, Auth.TokenValidationResult result,
    Boolean canCreateUser, String appDeveloperName, Auth.IntegratingAppType appType) {
    String identifier = result.getUserData().email; // or .username
    return [SELECT Id FROM User WHERE Azure_AD_Email_Id__c = :identifier
        AND IsActive = true LIMIT 1]; // never auto-provisions
}
```

Confirmed live against a real Teams SSO token: iss matches https://login.microsoftonline.com/<tenant>/v2.0 exactly; aud is the bare Client ID (not the Application ID URI) - both are still accepted in EXPECTED_AUDIENCES since Microsoft doesn't publicly document this as guaranteed behavior.

B.7 Troubleshooting log (the full journey to a working sign-in)

Symptom	Root cause	Fix
"App resource defined in manifest and iframe origin do not match"	The Application ID URI had no domain in it (api://<client-id> alone) - Teams checks that the domain portion of resource matches the tab's actual hostname	Set the Application ID URI to include that hostname: api://<tab-domain>/<client-id> (B.4)
"App webApplicationInfo or resource not defined in manifest" (appeared AFTER the fix above was made)	webApplicationInfo changes need a genuine uninstall + cache clear + reinstall, not a plain "update" - Teams was still running a stale cached manifest	Fully remove the app (not update), test via Teams web client first (less local caching than desktop), then re-upload as a fresh install
"FailedToOpenWindow" when the automatic fallback tried to open the popup	See Part A's troubleshooting log - the same browser popup-blocker issue, triggered here by the async getAuthToken() failure happening first	Same fix: a real "Sign in" button, popup only opened from its click handler

Symptom	Root cause	Fix
AADSTS500011: "resource principal ... was not found in the tenant"	getAuthToken() resolves the resource against the signed-in user's own home tenant; a mismatch between where the Azure AD app lives and that tenant, or a URI typo, produces this	Confirmed the Application ID URI matched exactly and the app registration was in the correct tenant
"token handler not found" (invalid_request)	OauthTokenExchangeHandler is a GLOBAL, org-wide component - confirmed by deploying every field the Metadata API would accept and finding no per-app link field at all. The actual link is a Setup-UI-only "Enable New App" step with no deployable metadata equivalent	Setup -> Token Exchange Handlers -> the handler -> Enable New App -> select the app, set a Run As user
"token handler validation failed" (generic, no detail)	Salesforce never surfaces the handler's real rejection reason to the caller, by design	Added temporary System.debug() logging inside the handler, retrieved the real ApexLog via the Tooling API after a live retry, then removed the logging once confirmed
Debug log: "Unauthorized endpoint ... Setup->Security->Remote site settings"	fetchPublicKey()'s JWKS callout to login.microsoftonline.com had no Remote Site Setting - unrelated to the token's actual validity	Added the Azure_AD_JWKS Remote Site Setting (B.5)
invalid_grant: "user hasn't approved this consumer" (after the handler itself started succeeding)	Token Exchange, like JWT Bearer flow, requires a prior approval that included a refresh_token grant on file - a non-interactive flow has no consent screen of its own to create that approval, so AllSelfAuthorized isn't sufficient	Permitted Users -> "Admin approved users are pre-authorized" + a Permission Set listed on the app's policy, assigned to the user (B.5)

Debugging technique worth keeping: pulling the real Apex debug log

Salesforce's OAuth error responses to the browser are deliberately generic ("token handler validation failed") and never explain why a custom handler rejected a token. The only way to see the real reason is a debug log: create a TraceFlag on the handler's Run As user (Tooling API subject TraceFlag, a DebugLevel id, a future ExpirationDate), have the person retry, then query ApexLog for Operation = 'OAuthTokenExchangeApexExec' and fetch its Body. This is how every Part B issue past the first two was actually diagnosed, not guessed.

C. How the two parts fit together

Nothing in Part A changed to make room for Part B - `loginWithTeamsSso()` was added as a new, independent first attempt, and `ensureSfSession()` decides which one actually ran a session:

```
ensureSfSession()
|-- session already in localStorage? -> use it (no network call at all)
|-- loginWithTeamsSso() (Part B, silent)
|   |-- succeeds -> session saved, done
|   |-- fails -> falls through, marks needsInteractiveSignIn
|-- caller shows a real "Sign in" button -> loginWithPopup() (Part A, interactive)
```

Both paths save the exact same session shape (`{accessToken, refreshToken, instanceUrl}`) to the same `localStorage` key (`sfUserSession`), so every function downstream of `ensureSfSession()` - `sfGetWithRetry`, `loadListView`, the object-tab picker, everything - is completely unaware of which path actually produced the session it's using.

D. FAQ

Q: Do people have to log in every time they open the app?

A: No. With Part B working, most people never see any login step at all, ever. Even without it (Part A alone), only the very first time - after that, a stored session is reused silently (with automatic background refresh) until it's explicitly signed out, the refresh token expires or is revoked (~365 days), or the browser's storage is cleared.

Q: Why does the fallback popup show a full address bar with Salesforce's/Microsoft's raw URL?

A: Deliberate, not a bug - Teams shows full browser chrome on any third-party OAuth popup so a malicious Teams app can never fake a chrome-less "Microsoft login" screen. Every third-party OAuth Teams integration works this way.

Q: Can this be done without the popup entirely, fully invisible?

A: Yes - this is exactly what Part B does, and it's now built and confirmed working, not just a researched option. It needed a second Azure AD app exposing an API, a new Apex class doing real JWT signature validation, and a change from `AllSelfAuthorized` to an admin-approved Permission Set model - a materially bigger build than Part A, which is exactly why Part A remains as the automatic fallback rather than being replaced.

Q: What happens for a brand new person who's never used this tab before?

A: If Part B's setup is complete for them (their user has the Permission Set assigned and `Azure_AD_Email_Id__c` populated), their very first sign-in is already silent. If not yet assigned, they see Part A's interactive popup instead, and their admin needs to assign them the Permission Set before Part B can work for them - Part A alone has no such per-person setup requirement.

Q: What happens if my company's Salesforce uses a different SSO provider (e.g. Okta)?

A: Part A's popup mechanism is `IdP`-agnostic - it simply renders whatever that org's real login page shows. The specific `Azure_AD_SSO` Auth Provider built here was needed only because this dev sandbox had no SSO configured at all; it should not be replicated as a competing, parallel login path in an org that already has a company-mandated SSO provider.

Q: Is the access token exposed in the browser a real security concern?

A: It's a known, deliberate trade-off ("Option B" in the project plan) - the token lives in the browser rather than behind a server, in exchange for a much smaller build with no relay-held secrets. Revisit only if this trade-off ever stops being acceptable for the sensitivity of the data involved; a relay/BFF-based alternative ("Option A") was scoped but not built.